SUMMER RESEARCH INTERNSHIP REPORT

Computer Log Anomaly Detection Using Machine Learning Techniques

A Comparative Study Using Machine Learning and Deep Learning Models

Submitted by

Rajol Kumar

Roll No.: 25MAC2R19

Under the Guidance of

Sri Vallabha Deevi

Adjunct Faculty

Department of Data Science and Artificial Intelligence

Wadhwani School of Data Science and Artificial Intelligence (WSAI)

Indian Institute of Technology Madras

Summer Research Internship

May–July 2026

Certificate

This is to certify that the internship report entitled **"Computer Log Anomaly Detection Using Machine Learning Techniques"**, submitted by **Rajol Kumar** (Roll No. 25MAC2R19), is a record of the research work carried out during the Summer Research Internship at the Wadhwani School of Data Science and Artificial Intelligence (WSAI), Indian Institute of Technology Madras.

The work was performed under the guidance of Sri Vallabha Deevi and focuses on the application of machine learning and deep learning methods to network-traffic anomaly detection.

Sri Vallabha Deevi
Adjunct Faculty
Department of Data Science and AI
Indian Institute of Technology Madras

Acknowledgement

I express my sincere gratitude to my mentor, Sri Vallabha Deevi, for providing me the opportunity to undertake this Summer Research Internship at the Wadhwani School of Data Science and Artificial Intelligence (WSAI), Indian Institute of Technology Madras.

His guidance, technical suggestions, regular discussions, and constructive feedback played an important role in improving both my understanding of anomaly detection and the methodology followed in this project. The internship gave me practical exposure to working with large network-security datasets, model evaluation under class imbalance, hyperparameter optimisation, and deep-learning-based anomaly detection.

I also thank the faculty members and internship coordinators of WSAI for providing an excellent learning environment and the resources required to carry out this work.

Finally, I am grateful to my family and friends for their continuous encouragement and support throughout the internship.

Rajol Kumar

Abstract

The increasing scale and complexity of computer networks have made automated intrusion detection an important cybersecurity requirement. Traditional signature-based intrusion detection systems are effective for previously observed attacks but are less suitable for identifying new or modified threats [1]. Machine-learning-based anomaly detection provides an alternative by learning patterns in network behaviour and identifying observations that deviate from those patterns [2].

This Summer Research Internship investigates supervised and unsupervised machine learning methods for network-traffic anomaly detection across datasets of increasing size and complexity. The first stage uses a small labelled network-traffic dataset to compare supervised classification with unsupervised anomaly detection. A leakage-safe train–test protocol is used, and preprocessing operations such as scaling, feature selection, class weighting, and SMOTE are restricted to the training process. The final tuned Support Vector Machine achieves an F1-score of 0.2243 and Recall of 0.6000. Among the unsupervised methods, the Autoencoder performs best with an F1-score of 0.1404.

The second stage evaluates supervised classification on the UNSW-NB15 benchmark [3, 4]. The official training and testing partitions are preserved. Logistic Regression, K-Nearest Neighbours, Linear SVM, Random Forest, XGBoost, LightGBM, and a Voting Classifier are compared. Mutual Information is used for feature selection and Optuna [5] for hyperparameter optimisation. LightGBM obtains the strongest validation performance and, after retraining on the complete official training partition, achieves an F1-score of 0.8921 and Recall of 0.9787 on the official test set.

A second medium-scale experiment uses the CSE-CIC-IDS2018 `02-15-2018.csv` file [6, 7]. Benign traffic is used for unsupervised model fitting, while DoS-GoldenEye traffic is treated as anomalous. Multivariate Gaussian modelling, Isolation Forest, One-Class SVM, and a Deep Autoencoder are studied under feature selection and hyperparameter optimisation. The optimized Autoencoder is selected for the large-scale study and achieves an F1-score of 0.8329 with Recall of 1.0000 on the unseen 5% anomaly holdout.

The large-scale experiment trains the Autoencoder on 300,000 benign flows and uses an

independent 30,000-flow benign calibration set. Three final holdout sets contain anomaly rates of 0.2%, 1%, and 5%. The tuned benign-only Autoencoder achieves F1-scores of 0.2548, 0.6322, and 0.8986 respectively. At the 5% anomaly rate it obtains Precision of 0.8171 and Recall of 0.9980.

Finally, robustness to contaminated training data is examined by introducing GoldenEye attacks into 300,000-record training sets at rates of 0.2%, 1%, and 5%. Performance deteriorates substantially as contamination increases. On the final 5% anomaly holdout, F1-score decreases from 0.8986 for benign-only training to 0.5129, 0.1131, and 0.0165 for 0.2%, 1%, and 5% contaminated training, respectively. The results demonstrate that while Deep Autoencoders can provide effective unsupervised network anomaly detection, their reliability depends strongly on the quality and purity of the normal reference data used for training.

Contents

Certificate 1

Acknowledgement 2

Abstract 3

1 Introduction 1

1.1 Introduction . . . 1
1.2 Computer and Network Logs . . . 1
1.3 Network Anomaly Detection . . . 2
1.4 Motivation . . . 2
1.5 Problem Statement . . . 3
1.6 Objectives . . . 3
1.7 Major Contributions . . . 4
1.8 Datasets Used . . . 5
1.9 Report Organization . . . 5

2 Literature Review 6

2.1 Intrusion Detection Systems . . . 6
2.1.1 Signature-Based Detection . . . 6
2.1.2 Anomaly-Based Detection . . . 6
2.2 Machine Learning for Intrusion Detection . . . 6
2.2.1 Supervised Learning . . . 7
2.2.2 Unsupervised and One-Class Learning . . . 7
2.3 Deep Learning for Network Security . . . 7
2.4 Autoencoder-Based Anomaly Detection . . . 8
2.5 Benchmark Intrusion Detection Datasets . . . 8
2.6 Research Gap Addressed . . . 9

3 Datasets and Methodology **10**

3.1 Experimental Philosophy . . . 10
3.2 Small Dataset . . . 10
3.3 UNSW-NB15 . . . 10
3.4 CSE-CIC-IDS2018 . . . 11
3.5 Missing and Infinite Values . . . 12
3.6 Feature Scaling . . . 12
3.7 Feature Selection . . . 12
3.7.1 Mutual Information . . . 12
3.7.2 Correlation Filtering . . . 12
3.8 Models . . . 13
3.8.1 Supervised Models . . . 13
3.8.2 Unsupervised Models . . . 13
3.9 Hyperparameter Optimisation . . . 14
3.10 Evaluation Metrics . . . 15

4 Small Dataset Experiments **16**

4.1 Experimental Setup . . . 16
4.2 Class Imbalance . . . 16
4.3 Feature Selection . . . 16
4.4 Supervised Model Comparison . . . 17
4.5 Final Tuned Supervised Model . . . 17
4.6 Unsupervised Models . . . 18
4.7 Discussion . . . 18

5 Medium Dataset Experiments **19**

5.1 Introduction . . . 19
5.2 Part A: UNSW-NB15 Supervised Classification . . . 19
5.2.1 Dataset Split . . . 19
5.2.2 Preprocessing . . . 19
5.2.3 Development Split . . . 20
5.2.4 Models . . . 20
5.2.5 Feature Selection . . . 20

5.2.6 Hyperparameter Optimisation . . . 21
5.2.7 Final Official Test Result . . . 21
5.3 Part B: CSE-CIC-IDS2018 Unsupervised Detection . . . 22
5.3.1 Dataset Preparation . . . 22
5.3.2 Training, Calibration, Development, and Final Data . . . 22
5.3.3 Models . . . 22
5.3.4 Experimental Configurations . . . 23
5.3.5 Feature Selection Result . . . 23
5.3.6 Representative Development Comparison . . . 23
5.3.7 Final Autoencoder Holdout Results . . . 24
5.4 Medium-Scale Discussion . . . 24

6 Large Dataset Experiments **26**
6.1 Introduction . . . 26
6.2 Dataset Cleaning . . . 26
6.3 Benign-Only Training Split . . . 27
6.4 Final Holdout Sets . . . 27
6.5 Baseline Autoencoder . . . 28
6.6 Optuna Optimisation . . . 28
6.7 Benign-Only Final Results . . . 29
6.8 Contaminated Autoencoder Training . . . 30
6.8.1 Motivation . . . 30
6.8.2 Data Separation . . . 30
6.8.3 Training Dataset Composition . . . 30
6.8.4 Preprocessing Under Contamination . . . 31
6.8.5 Threshold Calibration . . . 31
6.8.6 Development Results . . . 31
6.8.7 Final Contaminated Holdout Results . . . 32
6.8.8 Mean Final F1 . . . 33
6.8.9 Clean vs. Contaminated Training . . . 33
6.8.10 Interpretation . . . 33
6.8.11 Why Accuracy Is Misleading . . . 34
6.8.12 Operational Implication . . . 34

6.9 Chapter Summary 34

7 Conclusion and Future Work **36**

7.1 Conclusion 36

7.2 Limitations 37

7.3 Future Work 38

7.4 Final Remarks 39

List of Tables

1.1 Datasets used during the internship 5

3.1 Original CSE-CIC-IDS2018 label distribution 11

4.1 Small dataset supervised development results 17
4.2 Final tuned SVM performance 17
4.3 Unsupervised model performance on the small dataset 18

5.1 UNSW-NB15 official partitions 19
5.2 Best Optuna validation F1-scores on UNSW-NB15 21
5.3 Final LightGBM result on the UNSW-NB15 official test set 21
5.4 Final CSE-CIC-IDS2018 holdout composition 22
5.5 Medium CSE-CIC-IDS2018 experimental configurations 23
5.6 Representative best 5% development F1-scores 24
5.7 Optimized Autoencoder final holdout performance 24

6.1 Large-scale benign-only data allocation 27
6.2 Large-scale final holdout composition 27
6.3 Selected large-scale benign-only Autoencoder parameters 29
6.4 Large benign-only Autoencoder final results 29
6.5 Contaminated Autoencoder training sets 30
6.6 Final results for 0.2% contaminated training 32
6.7 Final results for 1% contaminated training 32
6.8 Final results for 5% contaminated training 32
6.9 Mean final F1 under contaminated training 33
6.10 Final comparison of clean and contaminated Autoencoder training 33

Chapter 1

Introduction

1.1 Introduction

Modern organisations depend heavily on interconnected computing systems. Financial services, cloud platforms, communication systems, educational institutions, healthcare infrastructure, and industrial networks generate very large volumes of digital activity every day. Although increased connectivity provides substantial benefits, it also creates a large attack surface for malicious activity.

Cyber attacks may attempt to interrupt services, obtain unauthorised access, steal information, install malicious software, or exhaust network resources. Detecting suspicious activity quickly is therefore an important component of cybersecurity.

Intrusion Detection Systems (IDS) monitor network or system activity and identify events that may indicate attacks. Traditional signature-based systems compare incoming activity with predefined attack patterns. Such systems work well for known threats but may fail when an attacker uses previously unseen or modified behaviour [1].

Machine learning offers an alternative approach in which patterns are learned directly from data [8]. In particular, anomaly detection attempts to learn the structure of normal behaviour and identify observations that differ substantially from it. This makes anomaly detection potentially useful for detecting attacks for which explicit signatures are unavailable.

1.2 Computer and Network Logs

Computer logs are automatically generated records describing events occurring in operating systems, applications, servers, databases, and network devices [9]. They are routinely used for monitoring, debugging, incident investigation, and security analysis.

The datasets used in this internship primarily contain *network-flow records*. A flow summarises communication between network endpoints using features such as:

- flow duration,
- source and destination ports,
- packet and byte counts,
- packet-rate and byte-rate statistics,
- inter-arrival times,
- protocol information,
- TCP flag statistics, and
- other statistical properties of communication.

Although these records are different from textual operating-system logs, they serve the same broader purpose of recording computer-system behaviour and are highly suitable for machine-learning-based security analysis.

1.3 Network Anomaly Detection

An anomaly is an observation whose behaviour differs substantially from the pattern expected under normal conditions [2]. In network security, anomalous flows may correspond to denial-of-service attacks, scanning activity, unauthorised access, malware communication, or other unusual network behaviour.

Anomaly detection is particularly useful when complete attack labels are unavailable. Instead of learning an explicit decision boundary between every known attack and benign traffic, a model may learn the distribution or representation of normal traffic and flag large deviations from it.

1.4 Motivation

Three practical issues motivate the work in this internship.

First, network-security datasets are usually highly imbalanced because malicious events may represent only a small fraction of total traffic. Accuracy can therefore be misleading.

Second, model performance can depend heavily on preprocessing and evaluation methodology. Leakage between training and testing data can produce unrealistically optimistic results.

Third, anomaly-detection models trained under the assumption of perfectly clean normal data may behave differently when the training set contains previously undetected attacks.

These issues motivated a progression from small-scale experimentation toward more rigorous and larger-scale evaluation.

1.5 Problem Statement

The central problem studied in this internship is:

> How effectively can supervised machine learning and unsupervised anomaly-detection methods identify malicious network traffic under different dataset sizes, class distributions, feature sets, and training-data compositions?

Particular emphasis is placed on Autoencoder-based anomaly detection, including its sensitivity to the presence of attack traffic in the training data.

1.6 Objectives

The objectives of the internship are:

1. Understand the application of machine learning to network anomaly detection.
2. Perform systematic exploratory analysis, data cleaning, feature preprocessing, and feature selection.
3. Compare supervised and unsupervised methods on a small labelled dataset.
4. Evaluate supervised classifiers on UNSW-NB15 while preserving the official train–test split.
5. Compare multiple unsupervised anomaly-detection algorithms on CSE-CIC-IDS2018.

6. Study the effect of feature selection and hyperparameter optimisation.
7. Evaluate a large-scale Deep Autoencoder using benign-only training.
8. Study Autoencoder robustness when the training data contains controlled attack contamination.
9. Evaluate models under anomaly prevalence levels of 0.2%, 1%, and 5%.
10. Use Precision, Recall, F1-score, False Positive Rate, and False Negative Rate in addition to Accuracy.

1.7 Major Contributions

The main contributions of this work are:

- A leakage-aware small-dataset comparison of supervised and unsupervised methods.
- A supervised UNSW-NB15 pipeline preserving the official training and testing partitions.
- A comparison of seven supervised classifiers with Mutual-Information feature selection and Optuna optimisation.
- A separate unsupervised study on CSE-CIC-IDS2018 using Multivariate Gaussian modelling, Isolation Forest, One-Class SVM, and Deep Autoencoders.
- Explicit separation of training, threshold calibration, hyperparameter-development, and final holdout data.
- Large-scale Autoencoder training on 300,000 benign network flows.
- Final testing on previously unseen 30,000-record holdout sets containing 0.2%, 1%, and 5% GoldenEye anomalies.
- A controlled contamination study using training contamination levels of 0.2%, 1%, and 5%.
- Quantitative analysis showing how contaminated training data can severely weaken reconstruction-based anomaly detection.

1.8 Datasets Used

Table 1.1: Datasets used during the internship

Dataset	**Scale**	**Raw Records**	**Learning Setting**	**Role**
Network Traffic Anomaly Detection Dataset	Small	1,000	Supervised and unsupervised	Initial model comparison and methodology development.
UNSW-NB15	Medium	257,673	Supervised	Official train–test classification benchmark.
CSE-CIC-IDS2018 `02-15-2018.csv`	Medium / Large	1,048,575	Unsupervised	Model comparison, large-scale Autoencoder training, and contamination analysis.

1.9 Report Organization

The report is organised as follows.

Chapter 2 reviews intrusion detection, supervised and unsupervised learning, and Autoencoder-based anomaly detection. Chapter 3 describes the datasets, preprocessing methodology, models, thresholding strategy, and evaluation metrics. Chapter 4 presents the small-dataset experiments. Chapter 5 presents the two medium-scale experiments: supervised classification on UNSW-NB15 and unsupervised anomaly detection on CSE-CIC-IDS2018. Chapter 6 presents the large-scale benign-only and contaminated Autoencoder experiments. Chapter 7 summarises the conclusions, limitations, and future work.

Chapter 2

Literature Review

2.1 Intrusion Detection Systems

An Intrusion Detection System monitors system or network activity with the objective of identifying malicious behaviour. IDS approaches are commonly grouped into signature-based and anomaly-based techniques [1].

2.1.1 Signature-Based Detection

Signature-based systems compare activity with previously defined rules or known attack patterns. Their principal strength is reliable detection of attacks already represented in the signature database. Their limitation is reduced effectiveness against zero-day or modified attacks for which a matching signature does not yet exist.

2.1.2 Anomaly-Based Detection

Anomaly-based detection models the expected behaviour of normal traffic and flags substantial deviations as suspicious. This approach does not require a predefined signature for every possible attack and can therefore help identify previously unseen threats.

Its main difficulty is that legitimate but unusual traffic may also look anomalous, leading to false positive alarms.

2.2 Machine Learning for Intrusion Detection

Machine learning algorithms learn patterns from data rather than being explicitly programmed with rules for every possible situation [8].

2.2.1 Supervised Learning

Supervised learning requires labelled examples. In intrusion detection, each training observation may be marked as normal or malicious. Common supervised methods include Logistic Regression, Support Vector Machines, Random Forest, XGBoost [10], and LightGBM [11].

When representative labelled attacks are available, supervised learning can provide strong classification performance. However, obtaining reliable labels for all possible attack types is difficult.

2.2.2 Unsupervised and One-Class Learning

Unsupervised or one-class anomaly detection is attractive when attack labels cannot be assumed. Models are trained primarily or exclusively on benign observations and identify unusual samples during inference.

Methods considered in this project include:

- Isolation Forest [12],
- One-Class Support Vector Machine [13],
- Multivariate Gaussian modelling,
- Gaussian Mixture Models,
- Elliptic Envelope,
- Hidden Markov Models, and
- Deep Autoencoders.

2.3 Deep Learning for Network Security

Deep learning can learn nonlinear relationships from high-dimensional datasets and has been widely investigated for cybersecurity applications [14]. Relevant architectures include Artificial Neural Networks, Convolutional Neural Networks, Recurrent Neural Networks, Long Short-Term Memory networks, and Autoencoders.

2.4 Autoencoder-Based Anomaly Detection

An Autoencoder is a neural network trained to reconstruct its input [15, 16]. It contains two principal components:

- an **encoder**, which maps the input into a compact latent representation; and
- a **decoder**, which reconstructs the original input from the latent representation.

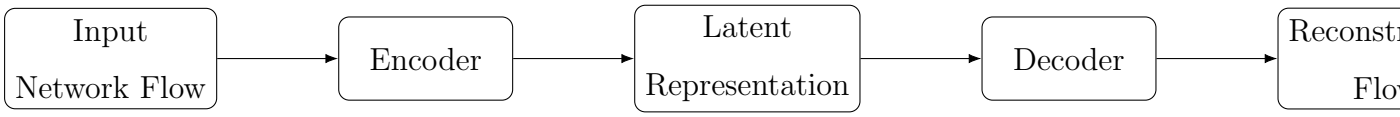


Figure 2.1: Conceptual Autoencoder architecture

If an Autoencoder is trained on benign traffic, it is expected to reconstruct familiar normal patterns with low error. Traffic that differs from the learned representation may produce larger reconstruction error.

For input vector $\mathbf{x}$ and reconstructed vector $\hat{\mathbf{x}}$, mean squared reconstruction error may be written as

$$E(\mathbf{x}) = \frac{1}{d}\sum_{j=1}^{d}(x_j - \hat{x}_j)^2. \tag{2.1}$$

An observation is classified as anomalous if

$$E(\mathbf{x}) > \tau, \tag{2.2}$$

where τ is a calibrated anomaly threshold.

2.5 Benchmark Intrusion Detection Datasets

Several datasets are widely used for intrusion-detection research, including KDD Cup 1999 [17], NSL-KDD [18], UNSW-NB15 [3], and CSE-CIC-IDS2018 [6].

Older datasets remain useful for comparison but may not fully represent modern network behaviour. UNSW-NB15 and CSE-CIC-IDS2018 contain more recent traffic characteristics and were therefore selected for the larger experimental stages of this work.

2.6 Research Gap Addressed

Many anomaly-detection experiments report a single performance value without examining:

- data leakage,
- differences between development and final testing,
- threshold calibration,
- anomaly prevalence,
- training-set contamination, or
- sensitivity to feature selection and hyperparameters.

This internship therefore focuses not only on model choice but also on experimental design. Separate training, calibration, development, and final holdout data are used where possible so that final reported performance better represents generalisation.

Chapter 3

Datasets and Methodology

3.1 Experimental Philosophy

An important objective of the revised experimental pipeline was to avoid leakage. Preprocessing operations that learn statistics from data were fitted using training data only and subsequently applied to validation or test data.

Final test sets were kept separate from model development wherever possible.

3.2 Small Dataset

The Network Traffic Anomaly Detection Dataset used for the initial experiments contains 1,000 records and 18 columns [19]. The target distribution contains approximately 90% normal and 10% anomalous observations.

The dataset contains numerical and Boolean traffic features. Boolean features were retained and converted to numerical representation. The feature `mean_packet_size` was constant and was removed.

A stratified 80:20 train–test split was performed before learned preprocessing operations.

3.3 UNSW-NB15

UNSW-NB15 contains an official training file with 175,341 observations and an official testing file with 82,332 observations. Both contain 45 columns.

The official split was retained throughout the experiment. This is important because combining the files before model development could leak information from the official test distribution.

The columns `id` and `attack_cat` were removed from the predictor set. The binary

`label` column was used as the target.

The categorical variables

- `proto`,
- `service`, and
- `state`

were one-hot encoded using a preprocessing pipeline fitted to training data.

The final encoded representation contained 194 features when fitted on the complete official training data.

3.4 CSE-CIC-IDS2018

The `02-15-2018.csv` file contains 1,048,575 rows and 80 columns.

The original label distribution is:

Table 3.1: Original CSE-CIC-IDS2018 label distribution

Class	**Records**
Benign	996,077
DoS attacks-GoldenEye	41,508
DoS attacks-Slowloris	10,990

Only Benign and DoS-GoldenEye traffic were retained for the main unsupervised experiments. Slowloris was excluded so that the study could focus on a single attack family.

After class filtering, the dataset contained 1,037,585 records.

Exact duplicate records were removed *before* removing the timestamp. This avoids treating otherwise identical flows occurring at different times as duplicates.

After removing 1,716 exact duplicates, the retained dataset contained 1,035,869 rows. The Timestamp feature was then removed.

The resulting binary data contained:

- 994,414 benign records; and
- 41,455 GoldenEye attack records.

3.5 Missing and Infinite Values

The raw CSE-CIC-IDS2018 file contained 4,921 missing values and 11,133 infinite values. The affected variables were primarily `Flow Byts/s` and `Flow Pkts/s`.

Infinite values were first replaced with missing values.

To avoid leakage, imputation statistics for model training were learned from the relevant training subset rather than globally from all data.

3.6 Feature Scaling

Different models have different scaling requirements.

- StandardScaler was used for algorithms such as Logistic Regression, SVM, and One-Class SVM where relative feature scale affects optimisation or distance.
- MinMaxScaler was used for the Autoencoder experiments.
- Isolation Forest was allowed to operate on imputed values without requiring distance-based scaling.

All scalers used for final evaluation were fitted using training data only.

3.7 Feature Selection

Two feature-selection ideas were investigated.

3.7.1 Mutual Information

For supervised classification, Mutual Information measures the dependence between each feature and the target class. It was used in the small dataset and UNSW-NB15 experiments.

For UNSW-NB15, the encoded feature set was reduced to the top 60 features for selected-feature experiments.

3.7.2 Correlation Filtering

For the unsupervised CSE-CIC-IDS2018 experiment, feature selection could not depend on attack labels without changing the learning setting. Therefore highly correlated features

were removed using only the benign training data.

Using an absolute correlation threshold of 0.95 reduced the feature set from 68 usable features to 43.

3.8 Models

3.8.1 Supervised Models

The following supervised models were investigated:

- Logistic Regression,
- Decision Tree,
- Random Forest,
- Support Vector Machine,
- K-Nearest Neighbours,
- XGBoost,
- LightGBM, and
- Voting Classifier.

Not every model was used on every dataset.

3.8.2 Unsupervised Models

The unsupervised experiments included:

- Multivariate Gaussian model,
- Gaussian Mixture Model,
- Isolation Forest,
- One-Class SVM,
- Elliptic Envelope,

- Hidden Markov Model, and
- Deep Autoencoder.

3.9 Hyperparameter Optimisation

Optuna was used for systematic hyperparameter optimisation [5]. Depending on the model, parameters included:

- number of estimators,
- tree depth,
- learning rate,
- subsampling ratios,
- kernel parameters,
- Autoencoder encoding dimension,
- hidden layer width,
- number of hidden layers,
- dropout,
- L2 regularisation,
- batch size, and
- target false-positive rate used during threshold calibration.

For the Autoencoder, development labels were used only to compare candidate configurations after unsupervised fitting. The model itself was trained by reconstruction and did not use attack labels as a supervised training target.

This distinction is important: the model-fitting procedure is unsupervised, while hyperparameter selection uses labelled development data to evaluate candidate detectors.

3.10 Evaluation Metrics

Because anomaly-detection datasets are highly imbalanced, Accuracy alone is inadequate.

Let:

- TP = true positives,
- TN = true negatives,
- FP = false positives,
- FN = false negatives.

Accuracy is

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}. \tag{3.1}$$

Precision is

$$\text{Precision} = \frac{TP}{TP + FP}. \tag{3.2}$$

Recall is

$$\text{Recall} = \frac{TP}{TP + FN}. \tag{3.3}$$

The F1-score is

$$F_1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}. \tag{3.4}$$

False Positive Rate is

$$\text{FPR} = \frac{FP}{FP + TN}. \tag{3.5}$$

False Negative Rate is

$$\text{FNR} = \frac{FN}{FN + TP}. \tag{3.6}$$

F1-score was treated as the primary summary measure, while Recall, FPR, and FNR were examined to understand operational trade-offs.

Chapter 4

Small Dataset Experiments

4.1 Experimental Setup

The small dataset was used to compare supervised learning with unsupervised anomaly detection.

A stratified 80:20 split was created before model preprocessing. Scaling, feature selection, SMOTE, and other learned transformations were restricted to the training process.

This approach avoids allowing information from the test data to influence training.

4.2 Class Imbalance

The dataset contains approximately 90% normal samples and only 10% anomalous samples.

Initial models could therefore achieve apparently strong Accuracy while predicting few or no attacks. Three approaches were considered:

- original imbalanced training,
- SMOTE applied inside the training pipeline [20], and
- class weighting.

4.3 Feature Selection

Mutual Information was used to estimate feature relevance using training data.

The strongest supervised cross-validation performance was obtained using four selected features.

4.4 Supervised Model Comparison

The development results using the selected feature set are shown in Table 4.1.

Table 4.1: Small dataset supervised development results

Model	Accuracy	Precision	Recall	F1
SVM	0.5650	0.1324	0.5875	**0.2157**
XGBoost	0.7787	**0.1602**	0.2875	0.2054
Logistic Regression	0.5450	0.1114	0.5125	0.1829
Random Forest	0.8238	0.1396	0.1375	0.1382
Decision Tree	0.8200	0.1022	0.1000	0.1000

The Support Vector Machine was selected for final tuning because it provided the strongest F1-score and high anomaly Recall.

4.5 Final Tuned Supervised Model

The tuned SVM was evaluated on the untouched test partition.

Table 4.2: Final tuned SVM performance

Metric	Value
Accuracy	0.5850
Precision	0.1379
Recall	**0.6000**
F1-score	**0.2243**

The relatively high Recall indicates that the SVM identifies a larger fraction of anomalous observations. However, its low Precision shows that this comes at the cost of many false alarms.

4.6 Unsupervised Models

Unsupervised models were trained using normal observations and evaluated on the held-out test data.

Table 4.3: Unsupervised model performance on the small dataset

Model	F1-score
Autoencoder	**0.1404**
Isolation Forest	0.1154
Gaussian Mixture Model	0.1111
Hidden Markov Model	0.1081
Elliptic Envelope	0.1071

The Autoencoder provides the strongest unsupervised result but remains below the final tuned SVM.

4.7 Discussion

The small dataset demonstrates several important lessons.

First, Accuracy is misleading under strong class imbalance. Several models obtained high Accuracy while detecting very few attacks.

Second, balancing strategies improve Recall but can reduce Precision substantially.

Third, feature selection improves the signal-to-noise ratio but does not make the problem easy. Even the best final supervised model reaches an F1-score of only 0.2243.

The results therefore motivated the use of larger and more realistic intrusion-detection benchmarks.

The implementation is available in the project repository [21].

Chapter 5

Medium Dataset Experiments

5.1 Introduction

The medium-scale stage contains two independent experiments.

The first uses UNSW-NB15 for supervised binary classification.

The second uses CSE-CIC-IDS2018 for unsupervised anomaly detection.

This separation avoids presenting results from fundamentally different learning settings as if they were directly comparable.

5.2 Part A: UNSW-NB15 Supervised Classification

5.2.1 Dataset Split

The official UNSW-NB15 partitions were retained:

Table 5.1: UNSW-NB15 official partitions

Partition	Normal	Attack	Total
Training	56,000	119,341	175,341
Testing	37,000	45,332	82,332

The official test data were not used for feature selection or hyperparameter optimisation.

5.2.2 Preprocessing

The following preprocessing pipeline was used:

1. Remove `id`.
2. Remove `attack_cat`.

3. Retain binary `label` as the target.
4. One-hot encode `proto`, `service`, and `state`.
5. Standardise numerical variables for scale-sensitive models.
6. Keep tree-based numerical features unscaled where appropriate.
7. Fit all preprocessing operations using training data.

No missing values or duplicate records were present in either official partition.

5.2.3 Development Split

An internal stratified validation partition was created from the official training data.

The official test set remained untouched until the final model had been selected.

5.2.4 Models

Seven models were studied:

- Logistic Regression,
- K-Nearest Neighbours,
- Linear SVM,
- Random Forest,
- XGBoost,
- LightGBM, and
- Voting Classifier.

5.2.5 Feature Selection

Mutual Information was calculated after encoding.

The selected-feature experiments used the top 60 encoded features, providing a compromise between dimensionality and validation performance.

5.2.6 Hyperparameter Optimisation

Optuna optimisation was applied to the strongest ensemble models.

Table 5.2: Best Optuna validation F1-scores on UNSW-NB15

Model	Best Validation F1
LightGBM	**0.9733**
XGBoost	0.9721
Random Forest	0.9714

LightGBM was selected for the final official test evaluation.

5.2.7 Final Official Test Result

After model selection, preprocessing and feature selection were refitted using the complete official training set. The tuned LightGBM model was then evaluated once on the official test partition.

Table 5.3: Final LightGBM result on the UNSW-NB15 official test set

Metric	Value
Accuracy	0.8696
Precision	0.8196
Recall	**0.9787**
F1-score	**0.8921**

The final test F1-score is lower than the internal validation F1-score. This difference illustrates why an untouched official test partition is valuable: it provides a more realistic estimate of model generalisation under a distribution that was not used during development.

The final classifier achieves very high attack Recall of 0.9787, meaning that most attack observations in the official test set are detected.

The complete implementation is available in [22].

5.3 Part B: CSE-CIC-IDS2018 Unsupervised Detection

5.3.1 Dataset Preparation

The CSE-CIC-IDS2018 experiment retained Benign and DoS-GoldenEye traffic.

After removing exact duplicates before dropping Timestamp, the dataset contained 994,414 benign records and 41,455 GoldenEye records.

The model training itself used benign traffic only.

5.3.2 Training, Calibration, Development, and Final Data

The medium-scale unsupervised experiment used:

- 100,000 benign flows for model fitting;
- 20,000 separate benign flows for threshold calibration;
- separate development/HPO data for model selection; and
- fresh final holdout data that had not been used for preprocessing, fitting, threshold calibration, feature selection, or hyperparameter optimisation.

The final holdout sets each contained 30,000 observations.

Table 5.4: Final CSE-CIC-IDS2018 holdout composition

Anomaly Rate	Benign	GoldenEye	Total
0.2%	29,940	60	30,000
1%	29,700	300	30,000
5%	28,500	1,500	30,000

5.3.3 Models

The principal models were:

- Multivariate Gaussian model,

- Isolation Forest,
- One-Class SVM, and
- Deep Autoencoder.

5.3.4 Experimental Configurations

Four broad configurations were investigated:

Table 5.5: Medium CSE-CIC-IDS2018 experimental configurations

Exp.	Description	Feature Selection	Optuna
1	All cleaned features, baseline parameters	No	No
2	Correlation-filtered features	Yes	No
3	All cleaned features with HPO	No	Yes
4	Correlation-filtered features with HPO	Yes	Yes

5.3.5 Feature Selection Result

Correlation filtering based only on benign training data reduced the 68-feature cleaned representation to 43 features.

This substantially improved the untuned Autoencoder. However, after hyperparameter optimisation, the strongest development performance was obtained by the Autoencoder using all cleaned features.

5.3.6 Representative Development Comparison

Representative best development results on the 5% anomaly development condition are shown below.

Table 5.6: Representative best 5% development F1-scores

Model	5% Development F1
Autoencoder, all features + Optuna	**0.8396**
One-Class SVM, selected + Optuna	0.5702
Isolation Forest, selected + Optuna	0.5584
Multivariate Gaussian	0.0462

These values refer to development/model-selection data, not the final holdout.

The Autoencoder was therefore selected for final evaluation and for the subsequent large-scale study.

5.3.7 Final Autoencoder Holdout Results

Table 5.7: Optimized Autoencoder final holdout performance

Anomaly Rate	Accuracy	Precision	Recall	F1	FPR	FNR
0.2%	0.9786	0.0856	1.0000	0.1577	0.0214	0.0000
1%	0.9789	0.3212	1.0000	0.4862	0.0213	0.0000
5%	0.9799	0.7136	1.0000	**0.8329**	0.0211	0.0000

On the 5% holdout, all 1,500 GoldenEye observations are detected, giving Recall of 1.0000. Precision of 0.7136 shows that the high attack Recall comes with some benign false positives.

The implementation is available in [23].

5.4 Medium-Scale Discussion

The two medium-scale experiments address different questions.

UNSW-NB15 demonstrates the effectiveness of supervised learning when attack labels are available. The final LightGBM classifier obtains F1 = 0.8921 on the official test partition.

The CSE-CIC-IDS2018 experiment addresses a more restrictive scenario in which model fitting is based on benign traffic. Among the candidate unsupervised methods, the Autoencoder provides the strongest development performance and achieves F1 = 0.8329 on the final 5% holdout.

The Autoencoder was therefore carried forward to the large-scale analysis.

Chapter 6

Large Dataset Experiments

6.1 Introduction

The large-scale study focuses exclusively on Deep Autoencoders.

Two questions are investigated:

1. How effectively does an Autoencoder trained on a substantially larger benign reference set detect GoldenEye attacks?
2. How does performance change when attack traffic contaminates the Autoencoder training data?

6.2 Dataset Cleaning

The same CSE-CIC-IDS2018 source file was used.

The cleaning sequence was:

1. Load 1,048,575 raw records.
2. Retain Benign and DoS-GoldenEye classes.
3. Remove exact duplicates while Timestamp is still present.
4. Remove 1,716 duplicate records.
5. Drop Timestamp.
6. Replace positive and negative infinite feature values with missing values.
7. Learn imputation statistics from the training data.
8. Remove constant features using training data.

9. Fit MinMaxScaler using training data.

After label filtering and duplicate removal, 1,035,869 records remained.

The final class pools contained:

- 994,414 benign flows; and
- 41,455 GoldenEye flows.

6.3 Benign-Only Training Split

The clean large-scale experiment used:

Table 6.1: Large-scale benign-only data allocation

Purpose	**Records**
Benign model training	300,000
Benign threshold calibration	30,000
Separate benign development pool	60,000
Separate benign final pool	90,000

Attack development and final-evaluation pools were also separated so that final attacks were unseen during model selection.

6.4 Final Holdout Sets

Three final sets were constructed, each containing 30,000 records.

Table 6.2: Large-scale final holdout composition

Anomaly Rate	**Benign**	**GoldenEye**	**Total**
0.2%	29,940	60	30,000
1%	29,700	300	30,000
5%	28,500	1,500	30,000

6.5 Baseline Autoencoder

The initial large-scale Autoencoder used a symmetric encoder–decoder architecture with a compressed latent representation.

Model training minimised Mean Squared Error, with Early Stopping based on validation reconstruction loss.

Threshold calibration used benign calibration data.

6.6 Optuna Optimisation

The large-scale Autoencoder search included:

- encoding dimension,
- hidden dimension,
- number of hidden layers,
- learning rate,
- dropout,
- L2 regularisation,
- batch size, and
- threshold target FPR.

The selected clean-training configuration included:

Table 6.3: Selected large-scale benign-only Autoencoder parameters

Parameter	Selected Value
Encoding dimension	8
Hidden dimension	192
Hidden layers	2
Learning rate	5.97×10^{-4}
Dropout	0.0818
L2 regularisation	1.34×10^{-7}
Batch size	2048
Target FPR	0.01238

6.7 Benign-Only Final Results

After hyperparameter selection, the tuned Autoencoder was evaluated once on the previously unseen final holdout sets.

Table 6.4: Large benign-only Autoencoder final results

Anomaly Rate	Accuracy	Precision	Recall	F1	FPR	FNR
0.2%	0.9883	0.1460	1.0000	0.2548	0.0117	0.0000
1%	0.9884	0.4622	1.0000	0.6322	0.0118	0.0000
5%	0.9887	0.8171	0.9980	**0.8986**	0.0118	0.0020

The mean F1-score across the three final holdouts is

$$\frac{0.2548 + 0.6322 + 0.8986}{3} = 0.5952. \tag{6.1}$$

The 5% anomaly condition provides the strongest F1-score because Precision increases as attacks become less rare. Recall remains approximately one across all three conditions.

At the most difficult 0.2% prevalence, only 60 attacks occur among 30,000 observations. Even a relatively small number of false-positive benign flows therefore strongly reduces

Precision.

The clean large-scale implementation is available in [24].

6.8 Contaminated Autoencoder Training

6.8.1 Motivation

A common assumption in reconstruction-based anomaly detection is that the model is trained using clean normal data.

In practical networks, however, training collections may include previously undetected attacks.

This experiment therefore evaluates the sensitivity of an Autoencoder to controlled GoldenEye contamination.

6.8.2 Data Separation

Development and final attack samples were reserved before contaminated training sets were constructed.

This prevents the same GoldenEye records from appearing in both training and final evaluation.

A separate benign pool was also reserved for the contaminated-training experiments.

6.8.3 Training Dataset Composition

All contaminated training sets contained exactly 300,000 observations.

Table 6.5: Contaminated Autoencoder training sets

Training Contamination	Benign	GoldenEye	Total
0.2%	299,400	600	300,000
1%	297,000	3,000	300,000
5%	285,000	15,000	300,000

The total sample size is therefore held constant while contamination level changes.

However, the contaminated-training experiments use a separately reserved benign pool rather than exactly the same 300,000 benign row identities as the clean experiment.

Consequently, the comparison strongly indicates the effect of contamination but should not be interpreted as a perfectly controlled causal experiment in which only attack contamination changes.

This is treated as a limitation of the present study.

6.8.4 Preprocessing Under Contamination

For each contamination level:

1. median imputation statistics were learned from that contaminated training set;
2. constant features were identified using that training set;
3. MinMaxScaler was fitted to that training set; and
4. the same learned preprocessing was applied to the relevant calibration, development, and final data.

This reflects a realistic situation in which contamination can affect both representation learning and preprocessing statistics.

6.8.5 Threshold Calibration

A trusted benign calibration set was used to select the reconstruction error threshold.

Optuna was allowed to search the target FPR used to determine the threshold quantile.

The use of a clean calibration set means that the experiment evaluates robustness of the learned Autoencoder representation under training contamination while still assuming access to a relatively small trusted benign reference set for threshold calibration.

6.8.6 Development Results

The rebuilt optimized 0.2% contaminated model achieved a mean development F1-score of approximately 0.3521.

At 1% training contamination, rebuilt-model mean development F1 decreased to approximately 0.0884.

At 5% contamination, rebuilt-model mean development F1 was only approximately 0.0099.

The strong degradation suggested that attack behaviour was increasingly being reconstructed by the Autoencoder.

6.8.7 Final Contaminated Holdout Results

No further hyperparameter or threshold changes were made after model development.

All contaminated models were evaluated on the common final holdout sets.

Table 6.6: Final results for 0.2% contaminated training

Test Rate	Accuracy	Precision	Recall	F1	FPR	FNR
0.2%	0.9694	0.0337	0.5167	0.0633	0.0297	0.4833
1%	0.9657	0.1478	0.5100	0.2292	0.0297	0.4900
5%	0.9489	0.4900	0.5380	0.5129	0.0295	0.4620

Table 6.7: Final results for 1% contaminated training

Test Rate	Accuracy	Precision	Recall	F1	FPR	FNR
0.2%	0.9800	0.0073	0.0667	0.0131	0.0182	0.9333
1%	0.9728	0.0409	0.0767	0.0533	0.0182	0.9233
5%	0.9367	0.1891	0.0807	0.1131	0.0182	0.9193

Table 6.8: Final results for 5% contaminated training

Test Rate	Accuracy	Precision	Recall	F1	FPR	FNR
0.2%	0.9703	0.0000	0.0000	0.0000	0.0278	1.0000
1%	0.9626	0.0024	0.0067	0.0035	0.0278	0.9933
5%	0.9244	0.0235	0.0127	0.0165	0.0276	0.9873

6.8.8 Mean Final F1

Table 6.9: Mean final F1 under contaminated training

Training Condition	Mean Final F1
0.2% contamination	0.2684
1% contamination	0.0598
5% contamination	0.0067

6.8.9 Clean vs. Contaminated Training

The most important large-scale comparison is shown in Table 6.10.

Table 6.10: Final comparison of clean and contaminated Autoencoder training

Training Condition	Mean Final F1	5% Precision	5% Recall	5% F1
Clean Benign	**0.5952**	**0.8171**	**0.9980**	**0.8986**
0.2% Contamination	0.2684	0.4900	0.5380	0.5129
1% Contamination	0.0598	0.1891	0.0807	0.1131
5% Contamination	0.0067	0.0235	0.0127	0.0165

6.8.10 Interpretation

The results demonstrate a strong relationship between training-data composition and Autoencoder anomaly detection.

With benign-only training, GoldenEye observations remain unfamiliar to the reconstruction model. Their reconstruction errors can therefore remain substantially different from benign traffic.

When GoldenEye observations are repeatedly presented during training, the Autoencoder is explicitly optimised to reconstruct those observations as well. Their reconstruction errors consequently become less useful as indicators of malicious behaviour.

This mechanism becomes increasingly severe as contamination rises.

At 0.2% training contamination, the detector retains some useful separation and reaches an F1-score of 0.5129 on the 5% final holdout.

At 1% contamination, Recall on the 5% final holdout falls to 0.0807 and F1-score to 0.1131.

At 5% contamination, Recall is only 0.0127 and F1-score only 0.0165. The False Negative Rate reaches 0.9873, meaning that nearly all attacks are missed.

6.8.11 Why Accuracy Is Misleading

The 5% contaminated model still reports Accuracy above 0.92 on all three final holdouts.

However, its attack Recall is close to zero.

This illustrates why Accuracy should never be interpreted alone in strongly imbalanced anomaly-detection problems.

6.8.12 Operational Implication

The contamination experiment suggests that a reconstruction-based detector should not automatically treat all historical traffic as normal training data.

Potential operational safeguards include:

- curated clean baseline periods,
- pre-filtering of suspicious training records,
- robust contamination-resistant representation learning,
- iterative removal of high-error samples,
- periodic revalidation of the reference dataset, and
- monitoring of changes in reconstruction-error distributions.

The contaminated-training implementation is available in [25].

6.9 Chapter Summary

The large-scale experiments provide the central result of the internship.

A Deep Autoencoder trained on 300,000 clean benign flows achieves strong attack detection on unseen final holdout data. At the 5% anomaly rate, it achieves Precision of 0.8171, Recall of 0.9980, and F1-score of 0.8986.

The contamination experiment reveals an important weakness of reconstruction-based anomaly detection. Performance decreases sharply as attack samples appear in the training data.

The 5% holdout F1-score decreases from 0.8986 under benign-only training to 0.5129, 0.1131, and 0.0165 under 0.2%, 1%, and 5% training contamination respectively.

The results therefore indicate that the quality and purity of the normal reference data are critical to reliable Autoencoder-based network anomaly detection.

Chapter 7

Conclusion and Future Work

7.1 Conclusion

This Summer Research Internship investigated machine learning and deep learning methods for network-traffic anomaly detection across datasets of increasing scale and realism.

The small-dataset experiment demonstrated that highly imbalanced security data can produce misleading Accuracy values. After leakage-aware preprocessing, feature selection, and hyperparameter tuning, the best supervised model was an SVM with F1-score 0.2243 and Recall 0.6000. The best unsupervised model was an Autoencoder with F1-score 0.1404.

The UNSW-NB15 experiment preserved the benchmark's official training and testing partitions. Tree-based ensemble models performed strongly during internal validation. LightGBM achieved the best validation F1-score and was selected for final evaluation. On the untouched official test partition it achieved Accuracy 0.8696, Precision 0.8196, Recall 0.9787, and F1-score 0.8921.

The second medium-scale experiment studied unsupervised detection on CSE-CIC-IDS2018. A leakage-aware protocol separated model fitting, threshold calibration, hyperparameter development, and final testing. Among the investigated anomaly detectors, the Autoencoder demonstrated the strongest development performance. On the final 5% anomaly holdout it achieved Precision 0.7136, Recall 1.0000, and F1-score 0.8329.

The large-scale experiment increased the benign Autoencoder training population to 300,000 network flows. The optimized model achieved F1-scores of 0.2548, 0.6322, and 0.8986 on final holdout sets containing 0.2%, 1%, and 5% attacks respectively. At the 5% anomaly rate it detected 99.8% of attacks while maintaining an FPR of approximately 1.18%.

The contamination study produced the most important robustness result. Introducing

GoldenEye traffic into Autoencoder training sharply reduced detection performance. On the common 5% anomaly holdout, F1-score decreased from 0.8986 for clean benign training to 0.5129 with 0.2% contamination, 0.1131 with 1% contamination, and 0.0165 with 5% contamination.

The result is consistent with the basic mechanism of reconstruction learning. An Autoencoder learns to reconstruct the data it repeatedly observes. If malicious behaviour occurs sufficiently often in the training set, it can become part of the learned representation and no longer produce reliably high reconstruction error.

The overall conclusions of the internship are therefore:

1. Evaluation methodology is as important as model selection. Train–test leakage can make security models appear substantially stronger than they are.
2. Accuracy alone is not appropriate for strongly imbalanced anomaly-detection tasks.
3. Supervised ensemble methods are highly effective when representative labelled attacks are available.
4. Deep Autoencoders can provide effective unsupervised anomaly detection when trained using representative benign traffic.
5. Hyperparameter and threshold selection have a substantial effect on anomaly-detection performance.
6. Final holdout evaluation is essential because development scores can be optimistic.
7. Autoencoder anomaly detection is highly sensitive to contamination of the normal training reference set.
8. Maintaining clean and representative normal reference data is a critical requirement for reconstruction-based intrusion detection.

7.2 Limitations

Several limitations should be considered when interpreting the results.

- The large-scale study focuses on one attack family, DoS-GoldenEye.

- Results on one CSE-CIC-IDS2018 capture day may not generalise directly to every network environment.
- The anomaly thresholds depend on the calibration distribution.
- Autoencoder optimisation exhibits stochastic variation across repeated neural-network training runs.
- Labels are used during hyperparameter-development evaluation, even though the anomaly models themselves are fitted unsupervised.
- The contaminated-training experiments use a separately reserved benign training pool rather than replacing rows within the exact clean 300,000-flow training sample. Therefore sample composition as well as attack contamination may contribute to some performance difference.
- The study is offline and does not measure real-time inference latency or throughput in an operational IDS deployment.

7.3 Future Work

The following extensions would strengthen the research further.

- Evaluate additional CSE-CIC-IDS2018 attack categories.
- Extend the experiments to CICIDS2017, TON_IoT, and other modern intrusion-detection datasets.
- Repeat contamination experiments using exactly the same clean base training sample and progressively replacing benign records with attacks. This would isolate contamination as the only changed variable.
- Repeat neural-network experiments using several random seeds and report means and confidence intervals.
- Investigate robust Autoencoders designed to tolerate contaminated training data.
- Explore Variational Autoencoders and denoising Autoencoders.

- Investigate Transformer-based and sequence-based network anomaly detection.
- Study adaptive or online threshold calibration under changing traffic distributions.
- Evaluate continual and incremental learning.
- Examine explainability methods such as feature-level attribution for anomalous reconstruction errors.
- Evaluate real-time inference performance and deployment cost.
- Explore ensemble systems combining reconstruction error, Isolation Forest scores, and supervised models where partial labels are available.

7.4 Final Remarks

The internship progressed from small-scale experimentation to large-scale, leakage-aware anomaly detection. The final results show both the potential and the limitations of Deep Autoencoders.

When trained using clean and representative benign traffic, the Autoencoder is capable of detecting GoldenEye attacks with very high Recall. However, the contamination experiments demonstrate that this performance should not be assumed to persist when malicious traffic is absorbed into the training reference set.

Reliable anomaly detection therefore depends not only on selecting a powerful machine-learning model, but also on careful data preparation, evaluation design, threshold calibration, and continuous monitoring of training-data quality.

Bibliography

[1] A. Khraisat, I. Gondal, P. Vamplew, and J. Kamruzzaman, "Survey of intrusion detection systems: Techniques, datasets and challenges," *Cybersecurity*, vol. 2, no. 20, 2019.

[2] V. Chandola, A. Banerjee, and V. Kumar, "Anomaly detection: A survey," *ACM Computing Surveys*, vol. 41, no. 3, pp. 1–58, 2009.

[3] N. Moustafa and J. Slay, "The UNSW-NB15 dataset for network intrusion detection systems," in *Military Communications and Information Systems Conference (MilCIS)*. IEEE, 2015.

[4] Australian Centre for Cyber Security (ACCS), UNSW Canberra, "UNSW-NB15 dataset," UNSW Research, 2015. [Online]. Available: https://research.unsw.edu.au/projects/unsw-nb15-dataset

[5] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, "Optuna: A next-generation hyperparameter optimization framework," in *Proceedings of the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining (KDD)*, 2019.

[6] I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani, "Toward generating a new intrusion detection dataset and intrusion traffic characterization," in *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.

[7] Canadian Institute for Cybersecurity, University of New Brunswick, "CSE-CIC-IDS2018 dataset," UNB CIC Datasets, 2018. [Online]. Available: https://www.unb.ca/cic/datasets/ids-2018.html

[8] C. M. Bishop, *Pattern Recognition and Machine Learning*. Springer, 2006.

[9] A. Oliner, A. Ganapathi, and W. Xu, "Advances and challenges in log analysis," *Communications of the ACM*, vol. 55, no. 2, pp. 55–61, 2012.

[10] T. Chen and C. Guestrin, "XGBoost: A scalable tree boosting system," in *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining (KDD)*, 2016.

[11] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu, "LightGBM: A highly efficient gradient boosting decision tree," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[12] F. T. Liu, K. M. Ting, and Z.-H. Zhou, "Isolation forest," in *Proceedings of the 8th IEEE International Conference on Data Mining (ICDM)*, 2008.

[13] B. Schölkopf, J. C. Platt, J. Shawe-Taylor, A. J. Smola, and R. C. Williamson, "Estimating the support of a high-dimensional distribution," *Neural Computation*, vol. 13, no. 7, pp. 1443–1471, 2001.

[14] M. A. Ferrag, L. Maglaras, S. Moschoyiannis, and H. Janicke, "Deep learning for cyber security intrusion detection: Approaches, datasets, and comparative study," *Journal of Information Security and Applications*, vol. 50, 2020.

[15] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

[16] F. Chollet, *Deep Learning with Python*. Manning Publications, 2018.

[17] S. J. Stolfo, W. Fan, W. Lee, A. Prodromidis, and P. K. Chan, "KDD cup 1999 data," UCI KDD Archive, Tech. Rep., 1999.

[18] M. Tavallaee, E. Bagheri, W. Lu, and A. A. Ghorbani, "A detailed analysis of the KDD CUP 99 data set," in *Proceedings of the IEEE Symposium on Computational Intelligence for Security and Defense Applications (CISDA)*, 2009.

[19] Kaggle, "Network traffic anomaly detection dataset," Kaggle Datasets, 2023, accessed: 2026. [Online]. Available: https://www.kaggle.com/datasets

[20] N. V. Chawla, K. W. Bowyer, L. O. Hall, and W. P. Kegelmeyer, "SMOTE: Synthetic minority over-sampling technique," *Journal of Artificial Intelligence Research*, vol. 16, pp. 321–357, 2002.

[21] R. Kumar, "Computer log anomaly detection – small dataset experiments," https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Small_Dataset.ipynb, 2026, gitHub repository.

[22] ——, "Computer log anomaly detection – unsw-nb15 experiments," https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Medium_Dataset_1.ipynb, 2026, gitHub repository.

[23] ——, "Computer log anomaly detection – cse-cic-ids2018 medium-scale experiments," https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Medium_Dataset_2.ipynb, 2026, gitHub repository.

[24] ——, "Computer log anomaly detection – large dataset benign-only autoencoder," https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Large_Dataset_Benign.ipynb, 2026, gitHub repository.

[25] ——, "Computer log anomaly detection – large dataset contaminated autoencoder training," https://github.com/RajolKumar2003/Computer-Log-Anomaly-Detection/blob/dataset-code/Large_Dataset_Contamination.ipynb, 2026, gitHub repository.